



CT 261

Database Management System Emergency Resource Management System

Course Learning Outcome: CLO 2 (Design conceptual, logical & physical database schemas)

PLO: PLO4 (Design/Development of Solutions)

Team Participation

The project was developed collaboratively:

- **Database Design:** Entity creation, constraints, normalization.
- **Backend Development:** Node.js with Express APIs.
- **Frontend:** HTML, CSS, JavaScript for forms and reports.
- **Testing & Workflow Validation:** End-to-end SOS request handling.

Group members:

Musfirah Shakil (CT-24051)

Kinza (CT-24056)

Falak Naz (CT-24059)

Ayesha Rauf (CT-24098)

Table of Contents

1. Organization Analysis

- Business Requirements
- Problem Statement
- Current System Limitations

2. Analysis and Design

- Conceptual Model (ERD Logic)
- Logical Model & Functional Dependencies
- Normalization Process (1NF to BCNF)

3. Physical Design

- Database Schema & Data Types
- Design Constraints

4. DBMS Techniques & Implementation

- DDL and DML operations used
- Relational Joins

5. System Workflow / Steps of solution

6. Limitations of the Adopted Approach

7. Sample Queries & Reports

8. Project Outcomes & Conclusion

1. ORGANIZATION & PROBLEM ANALYSIS

Business Requirements & Problem Statement

The "Emergency Resource Management System" was developed to address the critical need for coordinated disaster response. Current manual systems often suffer from data fragmentation, making it difficult to track victims, available rescuers, and resource distribution in real-time. The primary requirement was to create a centralized platform that bridges the gap between victims in distress and the organizations capable of providing aid.

Limitations of Current Systems

Existing methods often rely on paper-based logs or disconnected spreadsheets, leading to:

Delayed Response: No central link between a victim's location and available resources.

Resource Wastage: Lack of tracking for cost and quantity of items like medicine or food.

Poor Coordination: Difficulty in managing team leaders and rescuer skill sets across different organizations.

2. ANALYSIS AND DESIGN

2.1 Conceptual Model (ERD & EERD Diagram Logic)

The conceptual design focuses on the interaction between people (Users/Victims/Rescuers), entities (Organizations/Teams), and events (Disasters/SOS Requests).

Key Entities: User, Victim, Disaster, Location, Resource, Organization, Rescuer, Team, SOS Request, and Allocation.

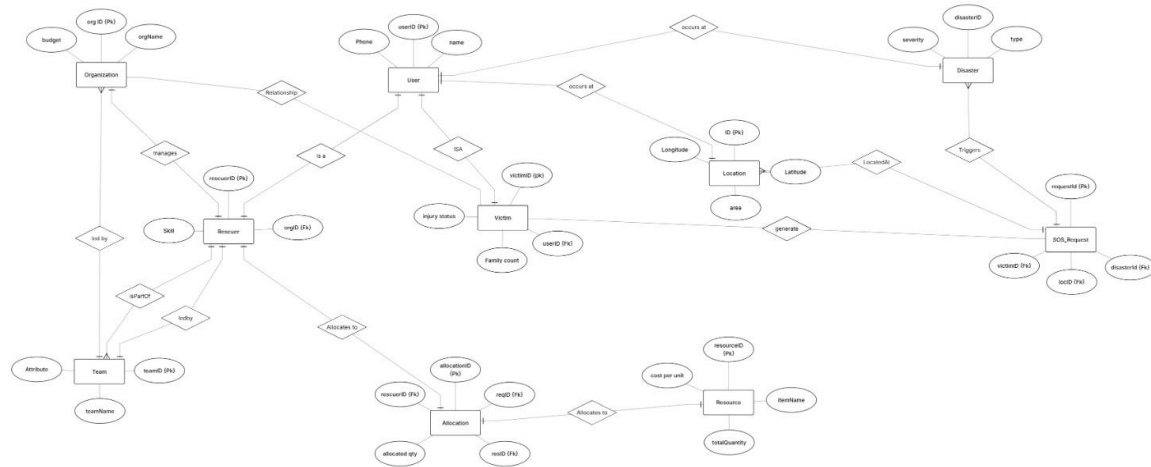
Business Rules: A User must exist before a Victim record is created. An SOS Request must link a Victim to a specific Disaster and Location.

. Relationships

- One User → One or Many Victims
- One Victim → Many SOS Requests
- One Disaster → Many SOS Requests
- One Location → Many SOS Requests
- One Organization → Many Rescuers
- One Rescuer → Can be Team Leader
- One SOS Request → Many Allocations

□ One Resource → Used in Many Allocations

ERD



Assumptions:

- (1) User $\rightarrow$ Victim (1:M) (e.g. family member)
One user can be recorded as multiple victims but each victim belongs to one user.
- (2) User $\rightarrow$ Rescuer (1:M)
One user can register as multiple rescuers but each rescuer belongs to single user.
- (3) Organization $\rightarrow$ Rescuer (1:M)
One organ. employs many rescuer but each rescuer is registered in exactly one organization.
- (4) Organization $\rightarrow$ Team (1:M)
An organ. manages multiple team but each team belong to 1 organization.
- (5) Rescuer $\rightarrow$ Team (1:1)
One rescuer can lead a team, but each team has led by only 1 leader.
- (6) Victim $\rightarrow$ SOS_request (1:M)
A single victim can raise many SOS request, but each request belongs to 1 victim.
- (7) SOS_request $\rightarrow$ location (1:M)
One location can have many SOS_request, but each request is for 1 loc.
- (8) SOS_request $\rightarrow$ allocation (1:M)
One SOS can have multiple allocation but each allocation is for 1 request.
- (9) Resource $\rightarrow$ allocation (1:M)
One resource type can be allocated many times, but each allocation refers to 1 resource.
- (10) Rescuer $\rightarrow$ allocation (1:M)
1 Rescuer can handle multiple allocation, but each allocation $\rightarrow$ 1 rescuer.
- (11) User occurs at Disaster, location (1:1) $\rightarrow$ Binary
Each user is tied exactly one disaster location and each location record corresponds to one user.

2.2 Logical Model & Functional Dependencies

The database has various Functional Dependencies;

Users ($\text{userId} \rightarrow \text{name, phone}$)

Victim ($\text{victimId} \rightarrow \text{injuryStatus, familyCount}$)

Resource ($\text{resourceId} \rightarrow \text{itemName, costPerUnit, totalQuantity}$)

Allocation ($\text{allocationId} \rightarrow \text{reqId, rescuerId, resId, allocatedQty}$)

2.3 Normalization

The database was designed using normalization principles to ensure eliminating data redundancy.

Normalization Process

Normalization:

1. User Table.

ID	Name	Phone
1	kinza	21346
2	falak	123456
3	musfirah	23
1003	TEST	123584
1004	Ayesha	457

① 1NF (First Normal Form)

Rule: No repeating groups, atomic values only.

Already Satisfied: Each cell has a single value.
(ID, Name, Phone) are atomic.

② 2NF (Second Normal Form)

Rule: No partial dependencies. (Non-key attribute depending on part of a composite key)

Already Satisfied: Primary key is ID (Single column), so no partial dependency possible.

① 3NF (Third Normal Form)

Rule: No Transitive Dependency (non-key dependent on another non-key)

Already Satisfied: Name & phone both depend only on ID, not on each other.

① BCNF (Boyce-Codd Normal Form)

Rule: Every determinant must be a candidate key

Already Satisfied: Only determinant is ID, which is the primary key.

2. Rescuer Table:

ID	Skill	Organization
1	FIRE FIGHTER	EDHI
2	DOCTOR	EDHI
3	DRIVER	RESCUE 112

1NF:

Rule: No repeating groups. Atomic Values only

Already Satisfied: Each cell has a single Value.

2NF:

Rule: No partial Dependency (non-key depending on part of composite key)

Already Satisfied: Primary key is ID (single col.), so no partial dependency.

3NF:

Rule: No transitive dependency.

Satisfied already: Skill and Organization both depend on ID, not on each other.

BCNF:

Rule: Every Determinant must be a Candidate key.

Satisfied Already: Only Determinant is ID, which is the primary key.

3. Location Table

ID	Longitude	Latitude	Area
1	192	22	Karachi
2	225	222	Lahore
3	2.25	2.25	gulshan
4	182	184	gulshan
1004	12.25	125	KARACHI DHA

① 1NF :

Rule: Atomic Values, No repeating Groups.

Satisfied: Each cell contains single value.

② 2NF :

Rule: No Partial Dependency.

Satisfied: Since ID is primary key (single column), so no Partial Dependency.

③ 3NF :

Rule: No Transitive dependency.

Satisfied already: Longitude, Latitude & Area depend directly on ID, not on each other.

④ BCNF :

Rule: Every determinant must be a candidate key.

Satisfied: Only determinant in this table is ID, which is the primary key.

4. Victim Table:

ID	User Name	Injury	Family
1	kinza	Severe	4
3	falak	heavy	5
1002	TEST	Critical	2

① 1NF:

Rule: No repeating groups. Atomic Values only.

Already Satisfied: Each cell has single value.

② 2NF:

Rule: No Partial Dependency.

Satisfied: PK is ID (single column), so no Partial dependency.

③ 3NF:

Rule: No Transitive dependency.

Satisfied: User Name, Injury & Family all depend directly on ID, not on each other.

④ BCNF:

Rule: Every determinant must be a candidate key.

Satisfied: Only determinant is ID, which is the Primary key.

5. Organization Table:

ID	Organization Name	Budget
1	EDHI	5000
2	RESCUE 112	6000

① 1NF :

Rule : Atomic Values, No repeating groups
Already Satisfied : Each cell has a single value.

② 2NF :

Rule : No Partial dependency
Satisfied : Primary key is ID (single column), so no Partial Dependency.

③ 3NF :

Rule : No Transitive dependency
Satisfied : Org. Name & Budget both depend directly on ID, not on each other.

④ BCNF :

Rule : Every determinant must be a candidate key.
Satisfied : Only Determinant is ID, which is the Primary key.

3. Physical Design

The physical implementation was built using SQL Server for the database and Node.js (Express) for the backend.

3.1 Database Schema & Data Types

We selected data types that balance storage efficiency with real-world precision.

Identification (Numeric): Every table uses an INT with the IDENTITY property as a Primary Key to ensure every record from a victim to a resource is unique.

Geographical Data (Floating Point): For precise mapping, the Location table uses FLOAT for coordinates, allowing us to pinpoint areas by latitude and longitude.

Financial Records (Decimal): To avoid rounding errors in relief funds, the Organization and Resource tables use DECIMAL for budgets and costs.

Descriptions (Alpha): Flexible VARCHAR fields are used for names, injury statuses, and skill sets, with character limits to prevent data bloating

3.2 Design Constraints

To prevent errors during a high-stress disaster scenario, the system enforces several "fail-safe" rules:

Referential Integrity: Foreign Keys act as a bridge; for example, you cannot assign a rescuer to a team unless that rescuer actually exists in the database.

Domain Validation: I implemented range constraints to keep data realistic, such as limiting familyCount to 100 or ensuring the severityLevel only accepts 'Low', 'Medium', or 'High'.

Mandatory Fields: Critical data, such as a victim's injuryStatus or a resource's totalQuantity, is marked as Required to ensure no incomplete or "blind" requests are processed.

Operational Logic: The system is structured so that an Allocation can only happen if there is a valid SOS Request and available Resources.

4. DBMS Techniques & Implementation

4.1 Data Definition language (DDL)

DDL queries define the structure of the database. These queries were used to create the tables, define data types, and establish primary/foreign key constraints as per our schema.

□ Creating Core Tables:

User Table :

```
CREATE TABLE Users (  
    userId INT IDENTITY(1,1) PRIMARY KEY,  
    name VARCHAR(100),  
    phone VARCHAR(20)  
);
```

Victim Table :

```
CREATE TABLE Victim (  
    victimId INT IDENTITY(1,1) PRIMARY KEY,  
    userId INT UNIQUE,  
    injuryStatus VARCHAR(100),  
    familyCount INT,  
  
    FOREIGN KEY (userId) REFERENCES Users(userId)  
);
```

Location Table :

```
CREATE TABLE Location (  
    locId INT IDENTITY(1,1) PRIMARY KEY,  
    longitude FLOAT,  
    latitude FLOAT,  
    area VARCHAR(100)  
);
```

□ Dropping Tables:


```
DROP TABLE IF EXISTS Resource_Allocation;
DROP TABLE IF EXISTS SOS_Request;
DROP TABLE IF EXISTS Rescuer;
DROP TABLE IF EXISTS Rescuer_Team;
DROP TABLE IF EXISTS Resource;
DROP TABLE IF EXISTS Victim;
DROP TABLE IF EXISTS Users;
DROP TABLE IF EXISTS Disaster;
DROP TABLE IF EXISTS Location;
DROP TABLE IF EXISTS Organization;
```

4.2 Data Manipulation Language (DML)

DML queries manage the actual data records. In the queries below, backend variables have been replaced with Sample Data for User to demonstrate how the system handles real-world scenarios.

□ Data Insertion (INSERT):

Insertion in User table:

```
32      await sql.query`
33          INSERT INTO Users (name, phone)
34          VALUES (${name}, ${phone})
35      `;
```

Insertion in Disaster table:

```
69      await sql.query`
70          INSERT INTO Disaster (type, severityLevel, reportedAt)
71          VALUES (${type}, ${severity}, GETDATE())
72      `;
73
```

Insertion in Victim table:

```

    await sql.query
    |
    |   INSERT INTO Victim (userId, injuryStatus, familyCount)
    |   VALUES (${userId}, ${injuryStatus}, ${familyCount})
    |   `;
    |
    |   res.send("Saved");
    |
    |   });

```

□ Data Retrieval (SELECT):

User table:

```

40  app.get('/getUsers', async (req, res) => {
41  |    let result = await sql.query(`SELECT * FROM Users`);
42  |    res.json(result.recordset);
43  |  });

```

Disaster table:

```

77  app.get('/getDisasters', async (req, res) => {
78  |    let result = await sql.query(`SELECT * FROM Disaster`);
79  |    res.json(result.recordset);
80  |  });

```

□ Data Updates (UPDATE):

User table:

```

49  await sql.query
50  |
51  |   UPDATE Users SET name=${name}, phone=${phone}
52  |   WHERE userId=${id}
53  |   `;

```

Disaster table:

```

86  await sql.query
87  |
88  |   UPDATE Disaster SET type=${type}, severityLevel=${severity}
89  |   WHERE disasterId=${id}
90  |   `;

```

□ Data Deletion (DELETE):

User table:

```
57 app.delete('/deleteUser/:id', async (req, res) => {
58   let id = req.params.id;
59   await sql.query`DELETE FROM Users WHERE userId=${id}`;
60   res.send("Deleted");
61 });
62
```

Disaster table:

```
94 app.delete('/deleteDisaster/:id', async (req, res) => {
95   let id = req.params.id;
96   await sql.query`DELETE FROM Disaster WHERE disasterId=${id}`;
97   res.send("Deleted");
98 });
99
```

4.3 Relational Joins

- **Victim and User Information:** This join connects the `Victim` table with the `Users` table to display the names of victims along with their injury status.

```
app.get('/getVictims', async (req, res) => {
  let result = await sql.query(`
    SELECT v.victimId, v.userId, u.name AS userName,
           v.injuryStatus, v.familyCount
    FROM Victim v
    INNER JOIN Users u ON v.userId = u.userId
  `);
}
```

- **SOS Request Mapping:**

This query links four different tables (`SOS_Request`, `Victim`, `Users`, `Disaster`, and `Location`) to provide a complete overview of each distress call.

```

✓ app.get('/report/full', async (req, res) => {
✓   let result = await sql.query(`
      SELECT s.requestId, u.name AS victimName,
      |      d.type AS disaster, l.area AS location, s.status
      FROM SOS_Request s
      INNER JOIN Victim v ON s.victimId = v.victimId
      INNER JOIN Users u ON v.userId = u.userId
      INNER JOIN Disaster d ON s.disasterId = d.disasterId
      INNER JOIN Location l ON s.locId = l.locId
    `);

```

□ Organization and Rescuer Coordination:

Connects the Rescuer table with the Organization table to identify which agency is managing specific skilled personnel.

```

app.get('/getRescuers', async (req, res) => {
  let result = await sql.query(`
    SELECT r.rescuerId, r.skillSet, o.orgName
    FROM Rescuer r
    INNER JOIN Organization o ON r.orgId = o.orgId
  `);

```

5. SYSTEM WORKFLOW /STEPS OF SOLUTION

To ensure data integrity and proper relational linking, the system follows a specific 11-step sequence

Step 1 → Add Users

User: Ali registers his basic contact details.

Step 2 → Create Victims from Users

Victim: A profile is created for Ali with his injury status.

Step 3 → Add Disaster

Event: A 'Flood' is logged with a High severity level.

Step 4 → Add Location

Geography: The location is set to 'Karachi DHA'.

Step 5 → Add Resources

Supply: 'Medicine' is added to the available inventory.

Step 6 → Add Organization

Agency: An NGO (e.g., Red Crescent) is registered.

Step 7 → Add Rescuers

Staff: A 'Doctor' is onboarded under the registered NGO.

Step 8 → Create Teams

Coordination: A specialized medical team is formed.

Step 9 → Create SOS Requests

Distress Call: Ali's need is linked to the Flood at DHA.

Step 10 → Allocate Resources

Deployment: The Doctor and Medicine are assigned to Ali.

Step 11 → Generate Reports

Audit: The system logs that the emergency was handled

6. LIMITATIONS OF THE ADOPTED APPROACH

Static Resource Tracking: While the system records allocations, it lacks an automated trigger to decrement totalQuantity in the Resource table when an allocation is made.

Manual Location Input: The system requires manual entry of Longitude and Latitude rather than utilizing live GPS API integration.

Connectivity Dependency: As a web-based solution, the system requires active internet/server access, which may be compromised during the peak of a natural disaster.

7.SAMPLE QUERIES & REPORTS

Full SOS Report: Shows victim, disaster, and location details.

```
✓ SELECT s.requestId, u.name AS victimName,  
       d.type AS disaster, l.area AS location, s.status  
FROM SOS_Request s  
INNER JOIN Victim v ON s.victimId = v.victimId  
INNER JOIN Users u ON v.userId = u.userId  
INNER JOIN Disaster d ON s.disasterId = d.disasterId  
INNER JOIN Location l ON s.locId = l.locId;
```

Geographical Crisis Distribution:

Helps identify "hotspots" by calculating the total number of SOS requests received from each geographical area.

```
SELECT l.area, COUNT(s.requestId) AS totalRequests  
FROM SOS_Request s  
INNER JOIN Location l ON s.locId = l.locId  
GROUP BY l.area;
```

Rescuer Workload:

Identifies how many tasks have been assigned to each rescue personnel based on their skill set.

```
SELECT r.skillSet, COUNT(a.allocationId) AS totalTasks  
FROM Resource_Allocation a  
INNER JOIN Rescuer r ON a.rescuerId = r.rescuerId  
GROUP BY r.skillSet;
```

Resource Utilization Summary:

Tracks the total quantity of relief items (like food, water, or medicine) that have been allocated to victims.

```
SELECT r.itemName, SUM(a.allocatedQty) AS totalUsed  
FROM Resource_Allocation a  
INNER JOIN Resource r ON a.resId = r.resourceId  
GROUP BY r.itemName;
```

Disaster Impact Analysis:

This query categorizes the total cases handled by the system according to the type of disaster (e.g., Flood, Earthquake, Fire).

```
SELECT d.type, COUNT(s.requestId) AS totalCases
FROM SOS_Request s
INNER JOIN Disaster d ON s.disasterId = d.disasterId
GROUP BY d.type;
```

Organizational Capacity Report:

Shows the total number of skilled rescuers contributed by each registered relief organization.

```
SELECT o.orgName, COUNT(r.rescuerId) AS totalRescuers
FROM Organization o
INNER JOIN Rescuer r ON o.orgId = r.orgId
GROUP BY o.orgName;
```

8. PROJECT OUTCOMES & CONCLUSION

Project Outcomes

The Emergency Resource Management System achieved the following outcomes:

Efficient Data Handling: DDL and DML operations ensured smooth creation, retrieval, updating, and deletion of records.

Integrated Reporting: Relational joins connected victims, disasters, locations, and resources into meaningful reports.

Resource Optimization: Allocation tables tracked usage of medicines, food, and rescuers, reducing wastage.

Workload Analysis: Reports highlighted rescuer tasks and organizational contributions for better planning.

Normalization: Up to BCNF, eliminating redundancy and ensuring data integrity.

Some UI/UX Snippets :

Victim Form

User:

kinza

Injury Status:

Enter injury status

Family Count:

Enter f:

Add

Save

Edit

Delete

Previous

Next

Records

ID	User Name	Injury	Family
1	kinza	severe	4
3	falak	heavy	5
1002	TEST	Critical	2

BASIC DATA

Users

Victim

Disaster

Location

Resource

RESCUE SETUP

Organization

Rescuer

Team

OPERATIONS

SOS Request

Allocation

REPORTS

Reports

Emergency Resource Management System

Users Form

Name:

kinza

Phone:

21346

Add

Save

Edit

Delete

Previous

Next

Records

ID	Name	Phone
1	kinza	21346
2	falak	123456
3	musfirah	23
1003	TEST	123584
1004	Ayesha	457

Emergency Resource Management System

SOS Request Form

Victim: -- Select Victim --

Disaster: -- Select Disaster --

Location: -- Select Location --

Add Save Edit Delete Previous Next

Records

ID	Victim Name	Disaster	Location
1	kinza	earth quake	karachi
2	falak	FIRE	gulshan
1002	TEST	Flood	KARACHI DHA

Emergency Resource Management System

Resource Allocation Form

SOS Request: -- Select Request --

Rescuer: -- Select Rescuer --

Resource: -- Select Resource --

Quantity:

Add Save Edit Delete Previous Next

Records

ID	Victim	Rescuer	Resource	Qty
1	TEST	DOCTOR	MEDICINE	50

Emergency Resource Management System

Rescuer Team Form

Team Name:

Leader:

Records

ID	Team Name	Leader (Skill)
----	-----------	----------------

Organization Form

Organization Name:

Budget:

Records

ID	Organization Name	Budget
1	EDHI	5000
2	RESCUE 1122	6000

Resource Form

Item Name:

Cost Per Unit:

Quantity:

Add Save Edit Delete Previous Next

Records

ID	Item	Cost	Quantity
1	water	500	5
2	food	50	60
1002	MEDICINE	50	5000

Reports:

Emergency Resource Management System

Reports

Requests per Location Full SOS Report Full Allocation Report Resources Used Rescuer Work Disaster Impact Organization Report

Reports

Requests per Location Full SOS Report Full Allocation Report Resources Used Rescuer Work Disaster Impact Organization Report

Location	Total Requests
gulshan	1
karachi	1
KARACHI DHA	1

Reports

Requests per Location

Full SOS Report

Full Allocation Report

Resources Used

Rescuer Work

Disaster Impact

Organization Report

ID	Victim	Disaster	Location
1	kinza	earth quake	karachi
2	falak	FIRE	gulshan
1002	TEST	Flood	KARACHI DHA

Reports

Requests per Location

Full SOS Report

Full Allocation Report

Resources Used

Rescuer Work

Disaster Impact

Organization Report

ID	Victim	Disaster	Location	Rescuer	Resource	Qty
1	TEST	Flood	KARACHI DHA	DOCTOR	MEDICINE	50

Reports

Requests per Location

Full SOS Report

Full Allocation Report

Resources Used

Rescuer Work

Disaster Impact

Organization Report

Resource	Total Used
MEDICINE	50

Reports

Requests per Location

Full SOS Report

Full Allocation Report

Resources Used

Rescuer Work

Disaster Impact

Organization Report

Rescuer Skill	Total Tasks
DOCTOR	1

Reports

[Requests per Location](#)[Full SOS Report](#)[Full Allocation Report](#)[Resources Used](#)[Rescuer Work](#)[Disaster Impact](#)[Organization Report](#)

Disaster	Total Cases
earth quake	1
FIRE	1
Flood	1

Conclusion

The Emergency Resource Management System is a practical prototype for disaster relief. It shows how database design and SQL queries can bring structure to chaotic situations by linking victims, rescuers, resources, and locations. Right now, the system has some limits for example, it doesn't yet have automatic inventory updates. But the foundation is strong and can grow further. With features like live GPS tracking and auto-generated reports, this system could become a complete solution for real-world emergencies. In short, it proves that a well-designed database can make disaster management faster, clearer, and more reliable.